



Facultad de Ingeniería y Ciencias Aplicadas

INGENIERÍA WEB

**Documento de
Especificación Funcional**

30-04-2026

UDLA:

Universidad de las Américas

Docente:

Juan José León Guerrero

Creado por:

Rodrigo Andrade

Contenido Plataforma Fullstack Business Intelligence y Gestión Operativa

1. Proceso de Diseño de Ingeniería Aplicado	4
1.1. Identificación del problema.....	4
1.2. Investigación y análisis del problema	4
1.3. Desarrollo de posibles soluciones	4
1.4. Selección de la mejor solución	5
1.5. Desarrollo de la solución	5
1.6. Pruebas y evaluación de la solución.....	5
1.7. Mejoras	6
2. Descripción	6
2.1. Plataforma Fullstack de Business Intelligence y Gestión Operativa:	6
2.2. Modelo de Gestión por Objetivos (MBO) y Motor MORC:	7
2.3. Sistema de Roles Operativos y Control Jerárquico:	7
2.4. Sistema MADi (Modo de Activación Dinámica Inteligente):	7
2.5. Arquitectura en Tiempo Real mediante WebSockets:	8
2.6. Interfaz Reactiva Basada en Eventos MADi:	8
3. Explicación del Core.....	8
3.1. Cálculo de Alcance Personal:.....	9
3.2. Cálculo de Utilidad Neta Real:	9
3.3. Multiplicador MADi:.....	9
3.4. Descuentos MADi:.....	10
3.5. Cálculo de Categorías de Rendimiento:.....	10
3.6. Cálculo de Bonos:.....	10
3.7. Sistema de Alertas MADi:	10
4. Alcance del Proyecto	11
4.1. Gestión Administrativa (Back-Office):.....	11
4.2. Capa de Comunicación en Tiempo Real Supabase Realtime:	12
4.3. Interfaz Operativa (POS y Gamificación):.....	13
4.4. Gestión de Piso (Supervisor):	14
4.5. Limitaciones del Sistemas	15
5. Seguridad de Datos y Control de Acceso	16
5.1. Bcrypt:.....	16
5.2. Supabase Auth y JSON Web Tokens (JWT):.....	16
5.3. RLS (Row Level Security en Supabase PostgreSQL):	17

5.4.	Jerarquía de Roles y Permisos:	17
5.5.	Recuperación Segura de Credenciales:.....	17
6.	Diagramas	18
6.1.	Diagrama Entidad-Relación (DER)	18
6.2.	Diagrama Entidad-Relación (DER)	19
6.3.	Diagrama de Clases Completo	20
6.4.	Diagrama de Casos de Uso	21
6.5.	Diagrama de Actividades (Flujo WebSockets + 5%)	22
7.	Wireframe - Blueprint.....	23
7.1.	LOGIN	23
7.2.	ADMIN.....	23
7.3.	SUPERVISOR	24
7.4.	MESERO	24
8.	Justificación del Stack Tecnológico	25
8.1.	Node.js (Express):.....	25
8.2.	Supabase (PostgreSQL):	25
8.3.	Vue.js:	25
8.4.	Tailwind CSS:	26
8.5.	Vite:.....	26
9.	Infraestructura y Despliegue Cloud	26
9.1.	Netlify (Frontend Deployment):	26
9.2.	Render (Backend Deployment):	27
10.	Análisis de Arquitectura: Principios SOLID y Patrón de Diseño	28
10.1.	Principios SOLID	28
10.2.	Patrones de Diseño Implementados	29
11.	Pruebas Funcionales del Sistema	32
11.1.	Prueba Funcional 1 (Patrón Observer): Sincronización de Mesas en Tiempo Real.	32
11.2.	Prueba Funcional 2 (Patrón Facade): Activación del Modo MADI / Happy Hour.	33

1. Proceso de Diseño de Ingeniería Aplicado

Para la evolución del proyecto "AromaGranoSystem", se aplicó rigurosamente la metodología de diseño de ingeniería de 7 pasos, partiendo de un sistema heredado (Legacy) desarrollado previamente en el semestre.

1.1. Identificación del problema

El sistema desarrollado previamente (Proyecto Core MVC) presentaba limitaciones operativas críticas para el entorno de una cafetería de alta demanda. Su arquitectura monolítica clásica generaba cuellos de botella: el Punto de Venta (POS) requería recargas de página completas (Page Reloads) para registrar un pedido o actualizar el estado de una mesa, impidiendo la colaboración concurrente entre meseros. Además, la lógica de negocio y la interfaz gráfica estaban fuertemente acopladas, dificultando el mantenimiento y violando el Principio de Responsabilidad Única (SRP).

1.2. Investigación y análisis del problema

Al analizar los cuellos de botella, se determinó que el problema raíz radicaba en el paradigma de comunicación síncrono del patrón MVC tradicional renderizado en servidor. Un entorno operativo de restaurante exige respuestas asíncronas e interfaces reactivas. Al depender de vistas renderizadas desde el servidor, la experiencia de usuario (UX) era deficiente en dispositivos táctiles (tablets de meseros), y carecía de mecanismos para implementar Patrones Observadores (Observer Pattern) que notificaran actualizaciones a otros clientes conectados.

1.3. Desarrollo de posibles soluciones

Para desvincular el frontend del backend y lograr la reactividad necesaria, se plantearon tres posibles vías arquitectónicas:

- **Alternativa A:** Mantener el monolito MVC e inyectar llamadas AJAX con jQuery para evitar recargas completas.
- **Alternativa B:** Separar el backend en una API REST (JSON) con Node.js y usar un framework de JavaScript del lado del cliente (Vanilla JS) manejando el DOM manualmente.
- **Alternativa C:** Implementar una arquitectura híbrida separando la persistencia en un BaaS (Backend as a Service), migrar el backend a un API REST pura en Node.js/Express, y consumir todo a través de una Single Page Application (SPA) reactiva usando el framework Vue 3.

1.4. [Selección de la mejor solución](#)

Se seleccionó la Alternativa C. Se justificó el uso de un framework JavaScript moderno (Vue.js 3 + Composition API) para consumir una API propia, ya que facilita la implementación del Patrón Observador nativo mediante WebSockets, permitiendo sincronización multiusuario. Además, la creación de una API JSON pura con Node.js facilita la aplicación del Principio de Inversión de Dependencias (DIP) y del Patrón Facade (Fachada) en los controladores.

1.5. [Desarrollo de la solución](#)

A continuación, se detallan los componentes de la solución implementada, incluyendo la arquitectura MVC+BaaS, el Módulo MORC, el POS reactivo y los principios de diseño aplicados.

1.6. [Pruebas y evaluación de la solución](#)

Una vez desarrollada la API y el aplicativo en Vue.js, se realizaron pruebas de concurrencia simulando múltiples meseros atacando el sistema POS de forma simultánea. La evaluación demostró que la separación de intereses (SoC) y la comunicación WebSocket eliminaron los conflictos de estado en

las mesas. El consumo del API JSON desde Vue.js redujo la latencia de operación a milisegundos. Finalmente, la plataforma fue desplegada (Deploy) exitosamente en entornos cloud de producción (Netlify para el Frontend y Render para el Backend API).

1.7. Mejoras

Tras la evaluación de la solución actual, se identificaron mejoras a implementar en futuras iteraciones:

- Refactorización profunda aplicando el Patrón Repositorio para independizar por completo las llamadas a la base de datos de los controladores.
- Implementación de caché local (PWA - Progressive Web App) en Vue.js para permitir que los meseros sigan tomando órdenes temporalmente ante microcortes de red.

Ampliación de los endpoints del API JSON para exponer reportes de Business Intelligence a herramientas de terceros como PowerBI.

2. Descripción

2.1. Plataforma Fullstack de Business Intelligence y Gestión Operativa:

El sistema Aroma & Grano es una plataforma fullstack de Business Intelligence y gestión operativa diseñada para cafeterías modernas que requieren centralización de procesos, monitoreo comercial en tiempo real y optimización táctica de sus operaciones diarias. El propósito principal del sistema es eliminar la fragmentación operativa existente entre la toma de pedidos, el control de mesas, el seguimiento comercial y la supervisión administrativa, integrando todos estos procesos dentro de una única arquitectura tecnológica conectada en tiempo real.

2.2. Modelo de Gestión por Objetivos (MBO) y Motor MORC:

La solución plantea un modelo de operación basado en Gestión por Objetivos (MBO), donde el rendimiento del personal no solamente se evalúa mediante ventas realizadas, sino también mediante indicadores estratégicos relacionados con utilidad neta, cumplimiento de metas y desempeño operativo. Para lograr esto, el sistema incorpora un núcleo analítico denominado MORC (Módulo de Optimización de Rendimiento Comercial), encargado de procesar métricas comerciales, calcular incentivos y administrar la lógica de gamificación aplicada al personal de atención.

2.3. Sistema de Roles Operativos y Control Jerárquico:

La plataforma contempla tres roles operativos claramente diferenciados: Administrador, Supervisor y Mesero. Cada uno posee permisos específicos, interfaces especializadas y restricciones de acceso definidas mediante mecanismos de seguridad avanzados. El Administrador tiene acceso a información financiera y configuración estratégica del negocio; el Supervisor monitorea el estado operativo de la cafetería y coordina al personal en tiempo real; mientras que el Mesero utiliza un Punto de Venta (POS) táctil diseñado para registrar pedidos y gestionar mesas de forma rápida y eficiente.

2.4. Sistema MADI (Modo de Activación Dinámica Inteligente):

Uno de los principales diferenciadores del sistema es la incorporación del sistema MADI (Modo de Activación Dinámica Inteligente), el cual funciona como un mecanismo de gamificación comercial y estrategia de ventas. Cuando las ventas acumuladas del día alcanzan determinados umbrales configurados por la administración, el sistema activa automáticamente un evento denominado "Happy Hour", aplicando descuentos dinámicos a los productos y otorgando multiplicadores virtuales de progreso a los

meseros. Esta lógica busca incentivar simultáneamente el volumen de ventas, la competitividad sana entre el personal y la experiencia operativa del negocio.

2.5. Arquitectura en Tiempo Real mediante WebSockets:

Adicionalmente, la plataforma incorpora sincronización en tiempo real mediante WebSockets, permitiendo que todos los dashboards, terminales POS y paneles operativos reflejen instantáneamente cualquier cambio relevante del sistema, incluyendo apertura de mesas, cierre de pedidos, progreso comercial, ranking de rendimiento y activación de eventos MADI. Esto permite que la cafetería funcione bajo un modelo de operación altamente reactivo, sincronizado y centralizado.

2.6. Interfaz Reactiva Basada en Eventos MADI:

Cuando el sistema activa el evento MADI (Happy Hour), todas las interfaces conectadas reaccionan visualmente en tiempo real mediante una transición automática hacia un “Modo Neón”. Este cambio modifica dinámicamente elementos visuales como bordes, botones, indicadores y banners utilizando acentos verdes brillantes para notificar al personal operativo que el multiplicador de progreso y los descuentos comerciales se encuentran activos. Esta retroalimentación visual inmediata mejora la coordinación operativa y fortalece la experiencia de gamificación dentro de la cafetería.

3. Explicación del Core

El núcleo lógico y analítico de la plataforma se encuentra representado por el MORC (Módulo de Optimización de Rendimiento Comercial). Este componente constituye el motor principal encargado de transformar datos

operativos en métricas estratégicas orientadas a la toma de decisiones comerciales y a la optimización del rendimiento del personal.

El MORC procesa continuamente información relacionada con ventas, costos, metas comerciales, progreso individual de los meseros, categorías de rendimiento y activación de eventos de gamificación. Todos los cálculos son ejecutados exclusivamente desde el backend para garantizar integridad matemática, consistencia operativa y protección contra manipulación de información desde el cliente.

El sistema implementa distintos cálculos estratégicos que permiten medir el rendimiento comercial del negocio y del personal operativo.

3.1. Cálculo de Alcance Personal:

$$Alcance\% = \left(\frac{Ventas\ Reales\ (VR)}{Meta\ Diaria\ (MD)} \right) \times 100$$

Este cálculo determina el porcentaje de cumplimiento de metas alcanzado por cada mesero. El resultado permite clasificar automáticamente al personal dentro de categorías comerciales como Bronce, Plata u Oro, dependiendo del rango configurado por administración.

3.2. Cálculo de Utilidad Neta Real:

$$UtilidadNeta = \sum(Venta) - \sum(CostoInsumos) - (SueldoBase + Bonos)$$

Esta fórmula permite calcular la rentabilidad real de la cafetería considerando ingresos generados, costos de producción, salarios base y bonos otorgados al personal. El indicador es utilizado por el dashboard administrativo para monitorear la salud financiera del negocio en tiempo real.

3.3. Multiplicador MADi:

$$VR\ Pedido = Subtotal \times 1.05$$

Cuando el sistema detecta que las ventas acumuladas del día superan el umbral MADi configurado, se activa automáticamente un

multiplicador virtual del 5% sobre el progreso comercial del mesero. Este incremento no altera el valor real pagado por el cliente; únicamente acelera el avance interno del trabajador hacia categorías y bonos superiores dentro del sistema de gamificación.

3.4. Descuentos MADI:

$$\text{PrecioMadi} = \text{PrecioBase} \times (1 - \text{PorcentajeDescuento})$$

Durante el modo Happy Hour, el sistema aplica automáticamente descuentos parametrizables sobre el catálogo de productos. Este porcentaje puede ser modificado por administración según las necesidades comerciales del negocio y busca incentivar el aumento del volumen de compra durante períodos de alta demanda.

3.5. Cálculo de Categorías de Rendimiento:

$$\text{Categoria} = f(\text{Alcance}\%)$$

El sistema clasifica automáticamente a los meseros dentro de categorías comerciales según el porcentaje de alcance obtenido. Por ejemplo, un rango entre 0% y 50% puede representar la categoría Bronce, entre 51% y 80% la categoría Plata, y valores superiores el nivel Oro. Estos rangos son totalmente configurables por administración.

3.6. Cálculo de Bonos:

$$\text{Bono} = \text{VentasAdicionales} \times \text{FactorBono}$$

Los bonos son calculados automáticamente considerando el excedente comercial alcanzado por cada trabajador y el factor porcentual definido por administración. Esta lógica permite implementar esquemas de incentivos dinámicos basados en productividad real.

3.7. Sistema de Alertas MADI:

$$\text{PorcentajeActual} = \frac{\text{VentasDia}}{\text{UmbralMADI}} \times 100$$

El sistema monitorea constantemente el progreso de ventas respecto al umbral MADi. Cuando las ventas alcanzan un porcentaje configurado previamente por administración, el Supervisor recibe una alerta visual indicando que el negocio se encuentra próximo a activar el modo Happy Hour. Esto le permite ejecutar estrategias comerciales anticipadas y preparar la operación para un incremento en la demanda.

El beneficio principal del sistema MADi consiste en combinar estrategias comerciales con elementos de gamificación operativa. Cuando el umbral configurado es alcanzado, la plataforma activa simultáneamente descuentos para los clientes y multiplicadores de progreso para los meseros, incentivando tanto el consumo como la competitividad sana dentro del equipo de trabajo.

4. Alcance del Proyecto

El sistema Aroma & Grano comprende una solución integral end-to-end que abarca desde la interacción táctica en el Punto de Venta (POS) hasta la generación de métricas estratégicas para análisis administrativo y financiero. La plataforma busca cubrir todas las necesidades operativas principales de una cafetería moderna mediante una arquitectura centralizada, reactiva y escalable.

4.1. Gestión Administrativa (Back-Office):

- El módulo administrativo constituye la capa estratégica y financiera del sistema. Desde este panel, el Administrador posee control completo sobre la configuración comercial, operativa y analítica de la cafetería.
- El sistema permite administrar el catálogo completo de productos, incluyendo precios de venta, costos de producción, control de stock y categorización de artículos. Esta información resulta fundamental para

el cálculo de utilidad neta y márgenes de contribución utilizados por el MORC.

- El Administrador puede configurar todos los parámetros relacionados con el sistema MADI y la gamificación operativa. Esto incluye la definición del umbral diario de ventas necesario para activar el modo Happy Hour, los porcentajes de descuento aplicados automáticamente a los productos, los multiplicadores de progreso utilizados para los meseros y las reglas asociadas a categorías de rendimiento y bonos comerciales.
- El dashboard administrativo incorpora herramientas de reportería estratégica capaces de visualizar información en tiempo real relacionada con ventas, utilidad neta, bonos pagados, productos más vendidos, rendimiento del personal y métricas comerciales relevantes para la toma de decisiones. Toda esta información puede ser exportada en formatos PDF, CSV o Excel para procesos contables y administrativos externos.
- El Administrador también posee control sobre la gestión de usuarios y credenciales del sistema. Puede crear cuentas, asignar roles, restringir permisos operativos y administrar accesos sensibles relacionados con el funcionamiento del negocio.

4.2. Capa de Comunicación en Tiempo Real Supabase Realtime:

- La plataforma incorpora una arquitectura de comunicación en tiempo real basada en Supabase Realtime y WebSockets, permitiendo sincronización instantánea entre todos los clientes conectados al sistema directo desde la base de datos PostgreSQL.
- Cada vez que ocurre un evento relevante dentro de la cafetería, como apertura de una mesa, cierre de un pedido, activación del MADI o

actualización de progreso comercial, el backend emite eventos en tiempo real hacia todos los dashboards y terminales POS conectados.

- Esta arquitectura elimina la necesidad de recargas manuales, garantizando que todos los usuarios visualicen información actualizada de manera inmediata. Gracias a esto, el Supervisor puede monitorear cambios operativos en tiempo real, mientras que el Administrador obtiene métricas comerciales instantáneas sin interrupciones.
- La utilización nativa de Supabase Realtime resulta especialmente importante para la activación del “Modo Neón” asociado al sistema MADI, ya que evita la necesidad de mantener un servidor de WebSockets externo (como Socket.io), permitiendo que todos los clientes reaccionen visualmente de forma inmediata cuando ocurre una alteración (UPDATE) en la tabla de configuración.
- Cuando la base de datos detecta una modificación o las ventas acumuladas alcanzan el umbral, **Supabase emite un broadcast global mediante WebSockets** hacia todos los clientes conectados. Este evento provoca que las interfaces del Administrador, Supervisor y Mesero cambien automáticamente hacia un estado visual denominado “Modo Neón”, reflejando en tiempo real que el sistema Happy Hour se encuentra activo.

4.3. Interfaz Operativa (POS y Gamificación):

- El sistema incorpora un Punto de Venta (POS) web optimizado para tablets, pantallas táctiles y dispositivos responsive. La interfaz fue diseñada priorizando rapidez operativa, facilidad de uso y mínima cantidad de interacciones necesarias durante la atención al cliente.
- El flujo operativo comienza cuando el mesero selecciona una mesa disponible desde el POS. Una vez registrada la primera orden, la mesa cambia automáticamente al estado “Ocupada” dentro del dashboard

del Supervisor. El pedido permanece activo durante toda la estancia del cliente, permitiendo agregar productos adicionales en cualquier momento sin necesidad de abrir nuevas órdenes.

- Cuando el cliente solicita la cuenta y realiza el pago, el mesero procede a cerrar el ticket desde el POS. Esta acción desencadena automáticamente múltiples procesos sincronizados: el sistema libera la mesa, actualiza las ventas acumuladas del día, recalcula el progreso comercial del mesero y sincroniza toda la información hacia los dashboards administrativos y operativos.
- La interfaz incorpora elementos de gamificación visual diseñados para incentivar el rendimiento comercial del personal. Cada mesero dispone de una barra de progreso reactiva que muestra su avance hacia metas comerciales configuradas por administración. Adicionalmente, el sistema muestra categorías visuales de rendimiento como Bronce, Plata y Oro, reforzando la motivación y competitividad sana dentro del equipo de trabajo.
- Cuando el modo MADI es activado, el POS cambia automáticamente hacia una interfaz estilo “Modo Neón”, incorporando bordes verdes brillantes, banners visuales y efectos reactivos asociados al evento Happy Hour. Esto permite que el personal identifique inmediatamente que el sistema de multiplicadores y descuentos se encuentra activo.

4.4. Gestión de Piso (Supervisor):

- El panel del Supervisor fue diseñado como una interfaz táctica enfocada en la coordinación operativa del piso de la cafetería. El sistema se encuentra optimizado para tablets y dispositivos touch, permitiendo monitoreo rápido y visual durante jornadas de alta demanda.

- El dashboard incorpora un mapa visual de mesas que permite identificar en tiempo real cuáles se encuentran libres y cuáles están ocupadas. Estos estados no son modificados manualmente por el Supervisor; en su lugar, son actualizados automáticamente según las acciones realizadas por los meseros desde el POS.
- Cuando un mesero abre un pedido asociado a una mesa, el sistema cambia automáticamente su estado a “Ocupada”. Posteriormente, cuando el ticket es cerrado y el cliente realiza el pago, la mesa vuelve automáticamente al estado “Libre”. Este mecanismo permite mantener consistencia operativa sin necesidad de intervención manual adicional.
- El Supervisor también posee acceso a indicadores relacionados con el rendimiento comercial del equipo, incluyendo rankings de meseros, progreso hacia metas MADI y alertas de proximidad al umbral Happy Hour. Cuando las ventas alcanzan determinados porcentajes configurados por administración, el sistema genera alertas visuales para que el Supervisor pueda aplicar estrategias comerciales oportunas.
- A diferencia del Administrador, el Supervisor no posee acceso a información financiera sensible como utilidad neta, costos de producción o configuración avanzada del sistema. Sus permisos se limitan estrictamente a monitoreo operativo y coordinación táctica del personal.

4.5. Limitaciones del Sistemas

- **Dependencia de Conectividad Constante:** Al basarse en Supabase Realtime (WebSockets) para la sincronización del estado de las mesas y la activación del MADI, el sistema requiere una conexión a internet estable. Las caídas de red deshabilitan la funcionalidad concurrente temporalmente.

- **Requisitos de Hardware en Cliente:** El despliegue del frontend en Vue 3 con reactividad avanzada y animaciones (Modo Neón) exige que los dispositivos táctiles (tablets o terminales POS) cuenten con navegadores modernos actualizados y memoria RAM suficiente (mínimo 2GB recomendados) para evitar cuellos de botella en el renderizado del DOM.

5. Seguridad de Datos y Control de Acceso

Para garantizar un sistema de nivel empresarial, se implementan rigurosas capas de validación y protección de la información:

5.1. Bcrypt:

Las contraseñas de todos los usuarios se almacenan mediante algoritmos de hashing con salting utilizando Bcrypt, garantizando que ninguna credencial pueda visualizarse directamente desde la base de datos. Esta estrategia protege al sistema frente a accesos indebidos, filtraciones de información y ataques de fuerza bruta, asegurando un manejo seguro de autenticación dentro de la plataforma.

5.2. Supabase Auth y JSON Web Tokens (JWT):

La autenticación de usuarios se implementa mediante Supabase Auth utilizando sesiones seguras basadas en JSON Web Tokens (JWT). Cada vez que un usuario inicia sesión, el sistema genera un token cifrado que permite validar identidad, permisos y accesos autorizados según el rol operativo correspondiente. Esta arquitectura facilita la protección de rutas privadas, el control de sesiones activas y la administración segura de credenciales dentro del ecosistema fullstack.

5.3. RLS (Row Level Security en Supabase PostgreSQL):

El sistema implementa políticas de seguridad a nivel de fila (RLS) sobre PostgreSQL mediante Supabase. Esta estrategia permite restringir información sensible dependiendo del rol autenticado. Gracias a estas políticas, los Meseros no pueden visualizar costos de producción, métricas financieras ni salarios del personal, mientras que los Supervisores únicamente acceden a información operativa relacionada con mesas, pedidos y monitoreo táctico. El acceso total a información financiera y configuraciones críticas queda reservado exclusivamente para el Administrador.

5.4. Jerarquía de Roles y Permisos:

La plataforma incorpora una jerarquía de permisos estructurada en tres niveles: Administrador, Supervisor y Mesero. Cada rol posee accesos específicos definidos desde backend, evitando modificaciones no autorizadas o accesos indebidos a funcionalidades críticas del sistema. El Administrador mantiene control absoluto sobre configuraciones estratégicas, mientras que Supervisor y Mesero operan únicamente dentro de sus responsabilidades operativas correspondientes.

5.5. Recuperación Segura de Credenciales:

El sistema implementa distintos mecanismos de recuperación de acceso dependiendo del rol del usuario. El Administrador y el Supervisor pueden recuperar sus contraseñas mediante Magic Links enviados al correo electrónico registrado. En contraste, los Meseros utilizan un sistema de restablecimiento rápido mediante PIN temporal generado directamente desde el panel del Supervisor o Administrador. Esta estrategia evita interrupciones operativas durante jornadas laborales de alta demanda.

6. Diagramas

6.1. Diagrama Entidad-Relación (DER)

Frontend Vue.js



Socket.io Client



Node.js API

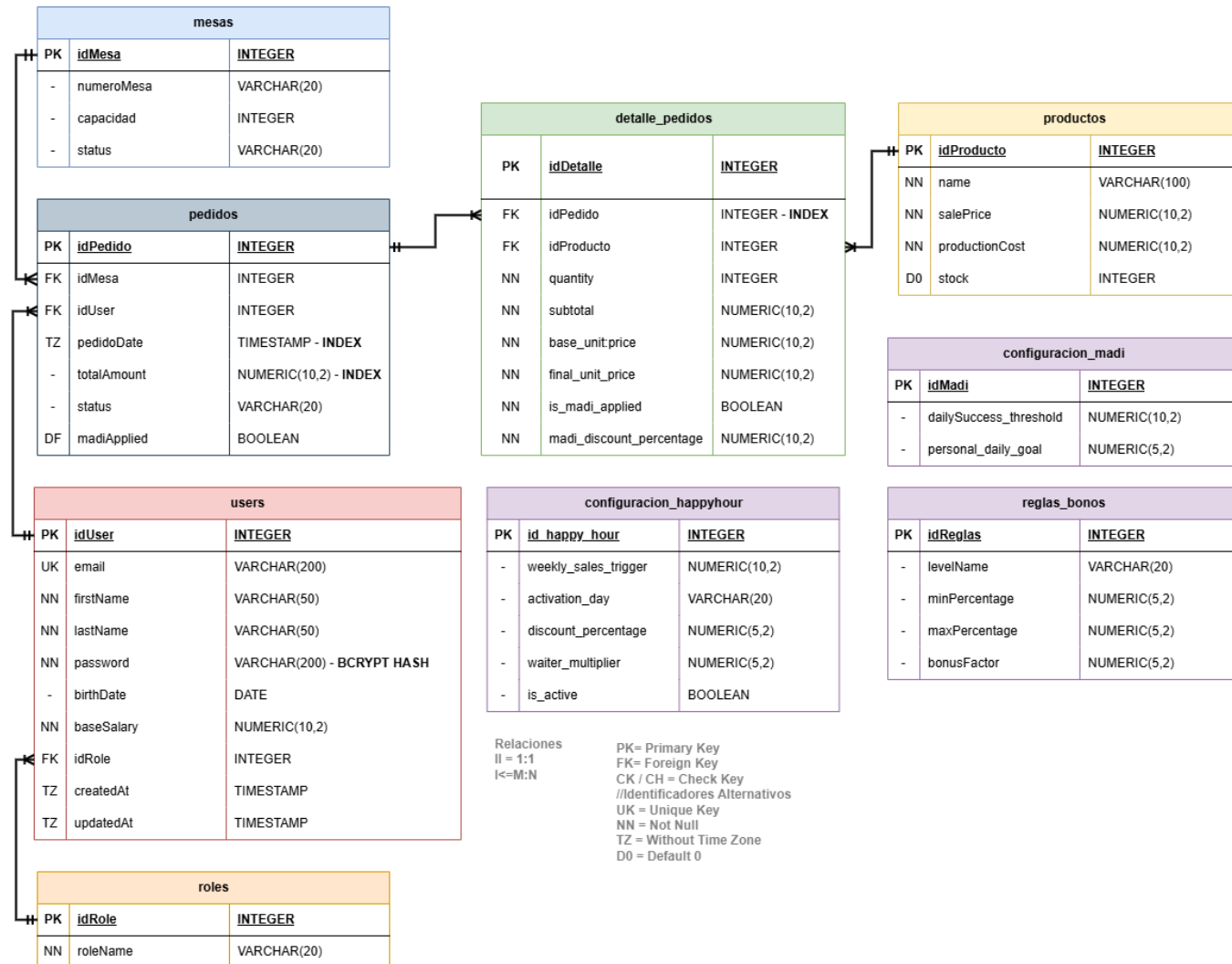


MORCCore

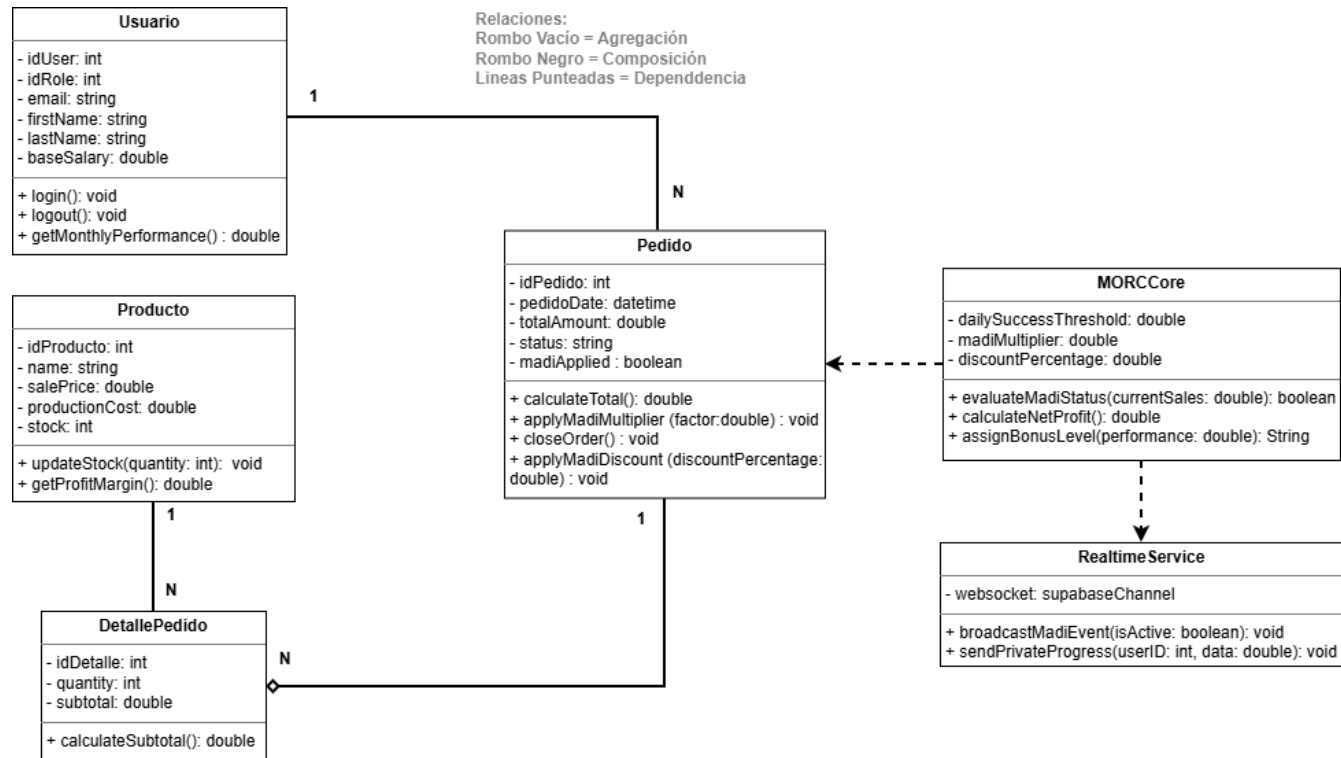


Supabase PostgreSQL

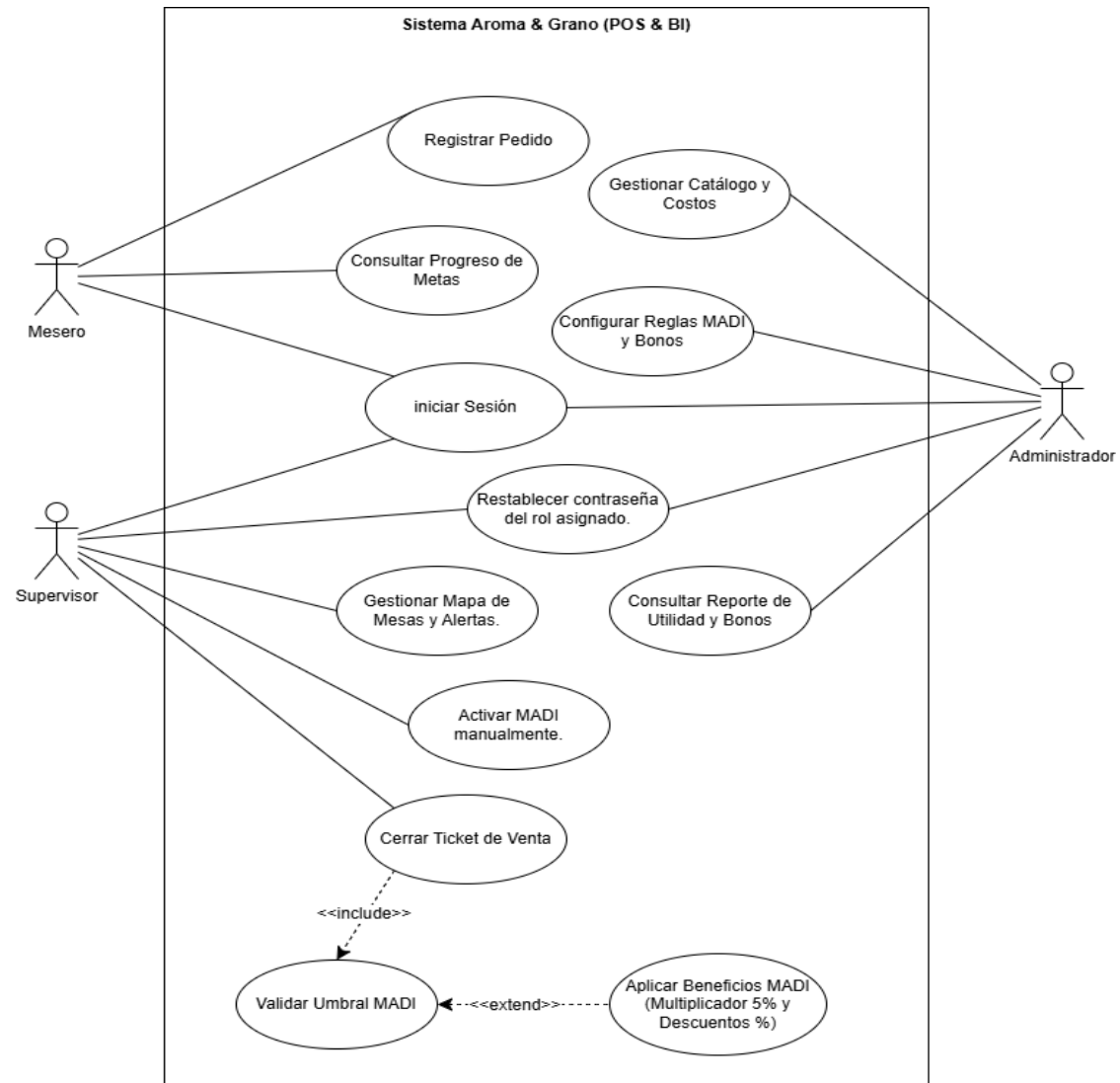
6.2. Diagrama Entidad-Relación (DER)

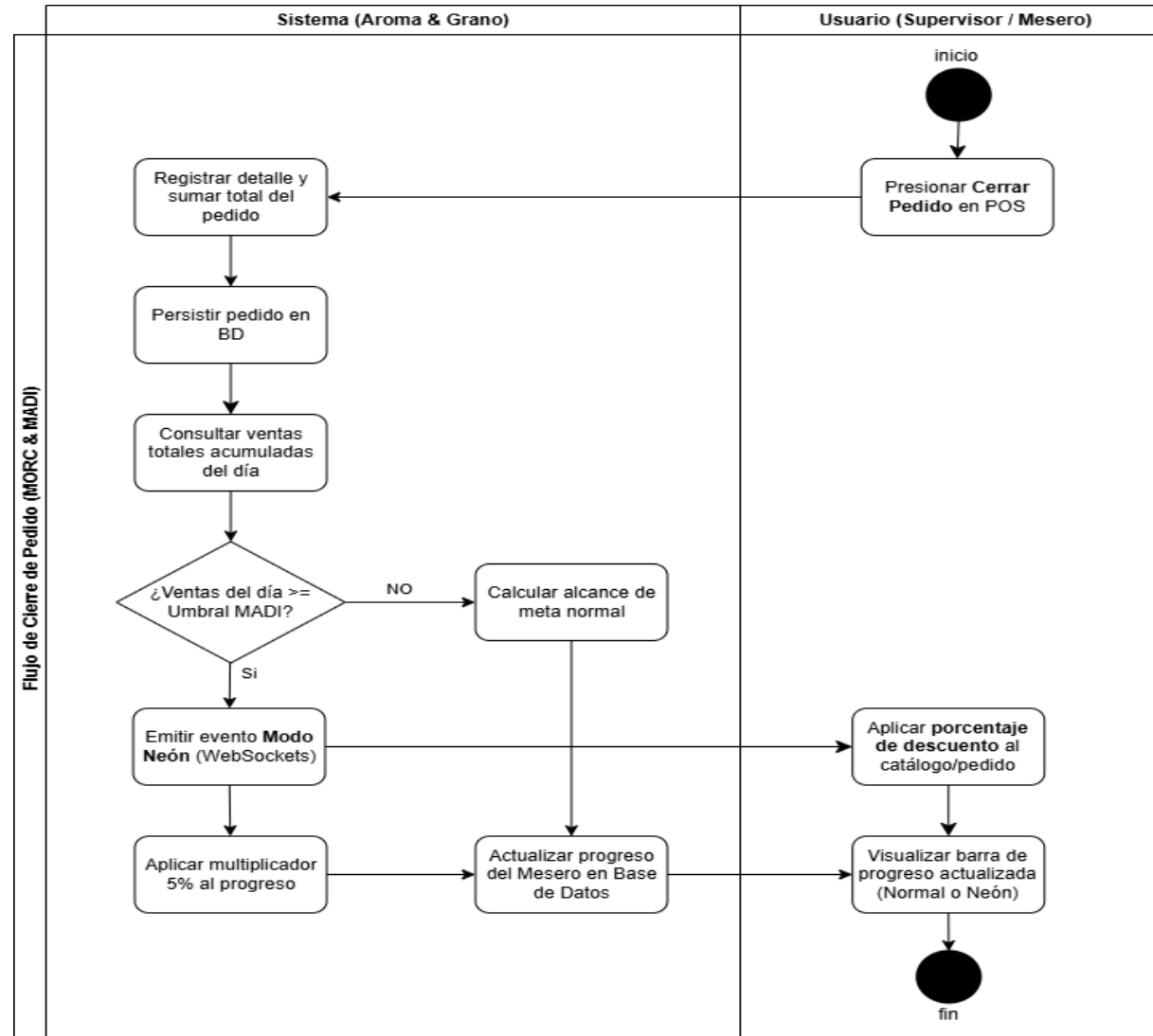


6.3. Diagrama de Clases Completo



6.4. Diagrama de Casos de Uso



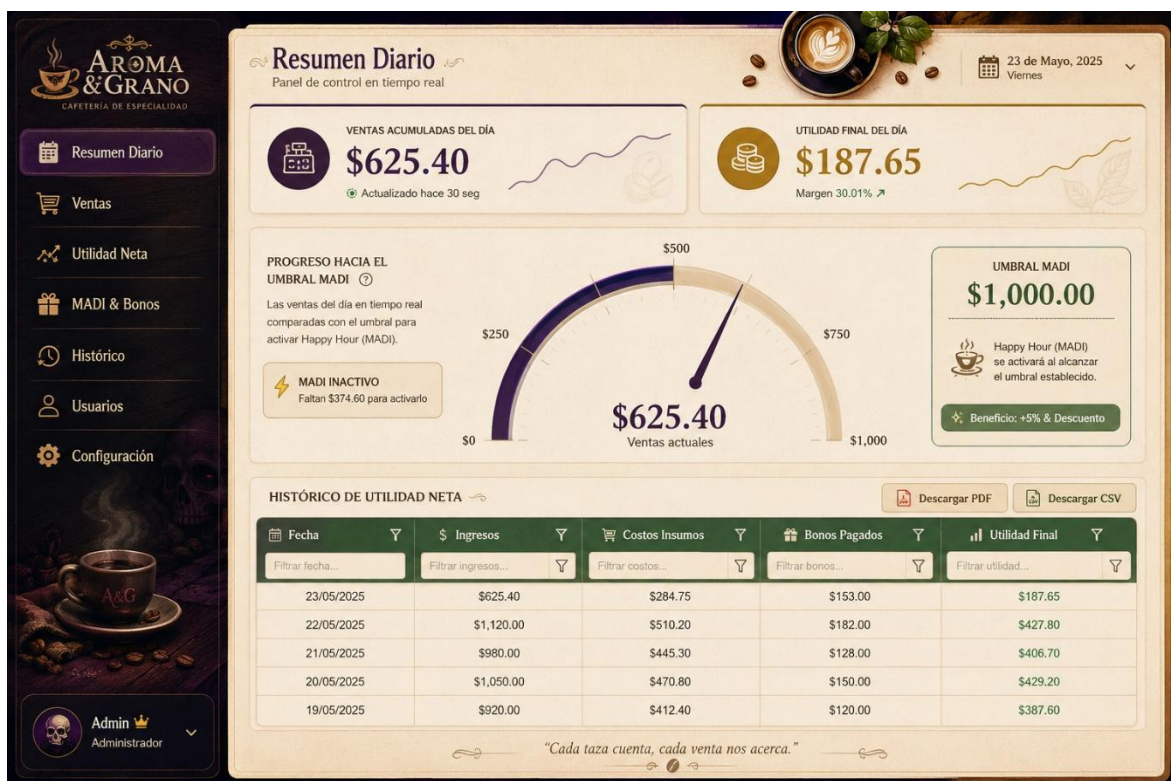
6.5. Diagrama de Actividades (Flujo WebSockets + 5%)

7. Wireframe - Blueprint

7.1. LOGIN



7.2. ADMIN



7.3. SUPERVISOR

Resumen Diario

Ventas

MADI & Bonos

Histórico

Usuarios

TURNO ACTUAL
Viernes, 23 de Mayo 2025
07:00 AM - 03:00 PM

SUPERVISOR
Diego Ramírez

HAPPY HOUR (MADI)

ACTIVAR

Happy Hour

Tiempo restante
01:24:36

ACTIVO

MAPA DE MESAS

1
Libre

2
Ocupada

3
Libre

4
Esperando cuenta

5
Ocupada

6
Libre

7
Ocupada

8
Libre

9
Esperando cuenta

10
Libre

11
Ocupada

12
Libre

● Libre

● Ocupada

● Esperando cuenta

ALERTAS DE SERVICIO

Mesa 4

Pedido #1025

Tiempo: 18 min

Ver detalle

Mesa 9

Pedido #1021

Tiempo: 15 min

Ver detalle

Mesa 7

Pedido #1027

Tiempo: 12 min

Ver detalle

Ver todas las alertas

PROGRESO HACIA EL UMBRAL MADI

72%

\$720.00

de \$1,000.00

VENTAS ACUMULADAS DEL DÍA
\$720.00

UMBRAL MADI
\$1,000.00

¡Felicidades! Happy Hour activo

Multiplicador +5% y descuentos aplicados

RANKING DE MESEROS (VENTAS DEL TURNO)

POS.	MESERO	VENTAS	% DE LA META
1	Juan Pérez	\$245.50	85%
2	María López	\$198.75	69%
3	Carlos Gómez	\$165.20	58%
4	Ana Torres	\$110.30	39%
5	Luis Martínez	\$95.30	33%

Ver ranking completo

Supervisor
Diego Ramírez

Cada taza cuenta, cada venta nos acerca.

7.4. MESERO

Meta diaria: \$95 / \$120

79%

¡HAPPY HOUR ACTIVO!

Multiplicador x1.05 +

AROMA & GRANO

CAFETERÍA DE ESPECIALIDAD

MODO NEÓN

Evento en vivo

CAFÉS

POSTRES

BEBIDAS FRÍAS

Espresso
\$1.60

Americano
\$2.20

Cappuccino
\$2.80

Latte
\$3.20

Mocha
\$3.60

Caramel Macchiato
\$3.80

Flat White
\$3.00

Cold Brew
\$3.50

Cheesecake
\$3.50

Brownie
\$2.90

Croissant
\$2.20

Frappé Caramelo
\$3.90

MESA ASIGNADA

Mesa 5

Cambiar mesa

ORDEN ACTUAL

1

+

Latte

\$3.20

\$3.20

x

2

+

Croissant

\$2.20

\$4.40

x

1

+

Cold Brew

\$3.50

\$3.50

x

1

+

Cheesecake

\$3.50

\$3.50

x

Agrega productos al ticket

Subtotal
\$14.60

Multiplicador Happy Hour (x1.05)
+\$0.73

TOTAL
\$15.33

ENVIAR ORDEN / COBRAR

La orden se enviará a cocina

Buscar producto

Personalizar

Notas

Descuento

Limpiar orden

11:42 AM

23 de Mayo, 2025

Conectado al sistema

Happy Hour activo

8. Justificación del Stack Tecnológico

8.1. Node.js (Express):

Constituye el núcleo principal del backend (Capa lógica y API REST) debido a su capacidad para manejar peticiones HTTP, concurrencia y eventos asíncronos de manera eficiente. Esta arquitectura resulta indispensable para procesar la lógica de negocio pesada, la validación de inventario y la ejecución segura de cálculos comerciales (MORC) sin comprometer credenciales en el frontend.

8.2. Supabase (PostgreSQL):

Fue seleccionado debido a su capacidad para garantizar integridad relacional, autenticación integrada y seguridad empresarial mediante **Row Level Security (RLS)**. La plataforma permite gestionar políticas de acceso granulares, asegurando que información sensible como costos de producción, salarios o métricas financieras solo sea visible para roles autorizados. Adicionalmente, la integración nativa con **JWT y Supabase Auth** simplifica considerablemente la implementación del sistema de autenticación y control de sesiones.

Adicionalmente, el módulo **Supabase Realtime** actúa como el núcleo de mensajería asíncrona mediante WebSockets directos, eliminando la necesidad de infraestructura Socket.io adicional y garantizando que las actualizaciones de estados y el Modo Neón se reflejen instantáneamente en los clientes.

8.3. Vue.js:

Fue elegido como framework frontend debido a su sistema de reactividad altamente eficiente y su arquitectura basada en componentes reutilizables. Esto facilita la construcción de dashboards dinámicos,

interfaces táctiles y actualizaciones visuales instantáneas provenientes de **WebSockets**. La transición reactiva hacia el “**Modo Neón**” asociado al sistema **MADI** puede implementarse fácilmente mediante cambios dinámicos de estado y estilos visuales controlados desde **Vue**.

8.4. [Tailwind CSS:](#)

Complementa la arquitectura frontend permitiendo desarrollar interfaces modernas, responsive y optimizadas para dispositivos táctiles. Su enfoque utility-first facilita la implementación rápida de diseños visuales complejos, incluyendo glassmorphism, efectos neón, dashboards modernos y componentes touch-friendly adecuados para entornos operativos reales dentro de cafeterías.

8.5. [Vite:](#)

Se implementa como herramienta moderna de construcción y desarrollo frontend debido a su capacidad para proporcionar **Hot Module Replacement (HMR)** extremadamente rápido. Esto optimiza significativamente el proceso de desarrollo y depuración de interfaces reactivas complejas, especialmente aquellas relacionadas con eventos visuales en tiempo real y lógica de gamificación dinámica.

9. [Infraestructura y Despliegue Cloud](#)

9.1. [Netlify \(Frontend Deployment\):](#)

Netlify será utilizado como plataforma de despliegue del frontend desarrollado en Vue.js. La elección de esta tecnología se debe a su integración nativa con aplicaciones SPA (Single Page Applications), su compatibilidad con Vite y su capacidad de realizar despliegues continuos automatizados directamente desde repositorios GitHub.

La plataforma permite implementar pipelines de Continuous Deployment (CD), donde cada actualización enviada al repositorio puede ser desplegada automáticamente en producción o entornos de prueba. Esto optimiza considerablemente el flujo de desarrollo del sistema Aroma & Grano.

- Adicionalmente, Netlify proporciona:
- Distribución global mediante CDN.
- Certificados SSL automáticos.
- Optimización de carga estática.
- Alta disponibilidad para interfaces en tiempo real.
- Compatibilidad responsive para acceso desde tablets, computadoras y dispositivos móviles.

Esta infraestructura resulta ideal para el frontend reactivo del sistema, especialmente considerando la existencia de eventos visuales dinámicos como el Modo Neón del MADi y la actualización instantánea de dashboards operativos.

9.2. Render (Backend Deployment):

Render será utilizado para el despliegue del backend API REST desarrollado en Node.js y Express. Esta plataforma permite ejecutar servicios persistentes y altamente disponibles, requisito indispensable para el funcionamiento de los controladores y la lógica de seguridad implementada en Aroma & Grano. Al delegar los WebSockets a Supabase, el backend en Render se optimiza exclusivamente para el procesamiento transaccional puro.

La elección de Render responde principalmente a:

- Compatibilidad estable con aplicaciones Node.js.

- Soporte continuo para conexiones WebSocket.
- Integración sencilla con variables de entorno seguras.
- Escalabilidad para servicios concurrentes.
- Despliegue automatizado desde GitHub.
- Monitoreo básico de logs y servicios backend.

El backend desplegado en Render será responsable de:

- Procesar autenticación JWT.
- Ejecutar la lógica del motor MORC.
- Calcular métricas comerciales.
- Gestionar activaciones MADI.
- Emitir eventos en tiempo real hacia supervisores y meseros.
- Sincronizar dashboards operativos y comerciales.

La combinación entre Netlify y Render permite mantener una arquitectura desacoplada, escalable y optimizada para aplicaciones fullstack modernas orientadas a eventos en tiempo real.

10. Análisis de Arquitectura: Principios SOLID y Patrón de Diseño

10.1. Principios SOLID

- **Separación de Intereses (SoC – Separation of Concerns)**

Implementación: El sistema ha sido desacoplado de manera que cada módulo tiene una única razón para cambiar.

- **Capa de Presentación (Vue.js):** Archivos como MeseroPOS.vue o SupervisorDashboard.vue se encargan estrictamente de renderizar la interfaz de usuario y reaccionar a los clics.

- **Capa de Estado (Pinia):** Archivos como happyHourStore.js aíslan y manejan globalmente variables como el estado del MADI.
- **Capa de Servicios:** Archivos como pedidosService.js abstraen por completo el consumo de la API REST. El componente visual no sabe cómo se comunica con la base de datos, solo llama a la función de servicio.
- **Principio de Responsabilidad Única (SRP - Single Responsibility Principle)**

Implementación: En el backend desarrollado en Node.js, la lógica no se programó en un solo archivo masivo. La aplicación está dividida en:

- **routes:** Definen únicamente los endpoints (ej. /api/pedidos).
- **controllers:** Reciben la petición HTTP, validan y devuelven respuestas JSON.
- **services:** Ejecutan la lógica pesada transaccional y se conectan a la base de datos (PostgreSQL).

10.2. Patrones de Diseño Implementados

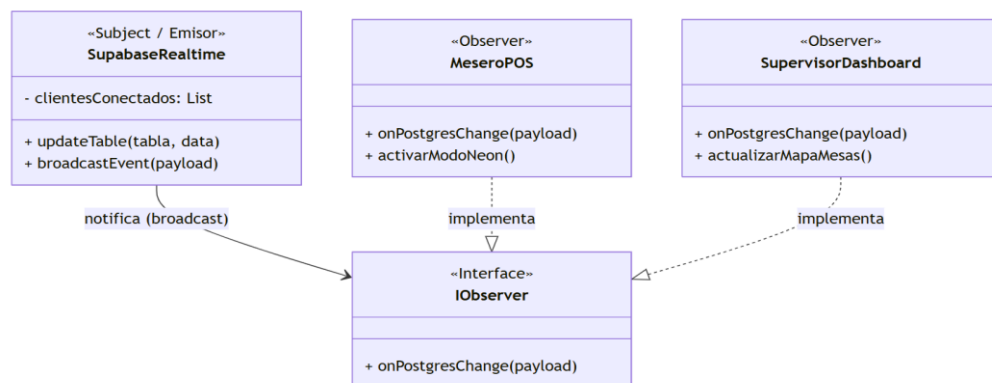
- **Patrón Observer (Observador) mediante WebSockets)**

Justificación: El sistema requiere que múltiples usuarios (Meseros, Supervisores) estén sincronizados en tiempo real sin recargar la página. **Implementación:** Se utilizó Supabase Realtime, el cual es una implementación nativa del patrón Observer.

- **El Sujeto (Subject):** Es la base de datos PostgreSQL. Cuando el backend actualiza una tabla (ej. configuracion_happy_hour o mesas), el motor emite un evento de cambio (broadcast).

- **Los Observadores (Observers):** Son los clientes Vue.js conectados (ej. pantallas de los meseros). Al estar "suscritos" al canal (`.subscribe()`), reaccionan automáticamente al evento y actualizan su estado (activando el "Modo Neón" o cambiando el color de una mesa a ocupada).

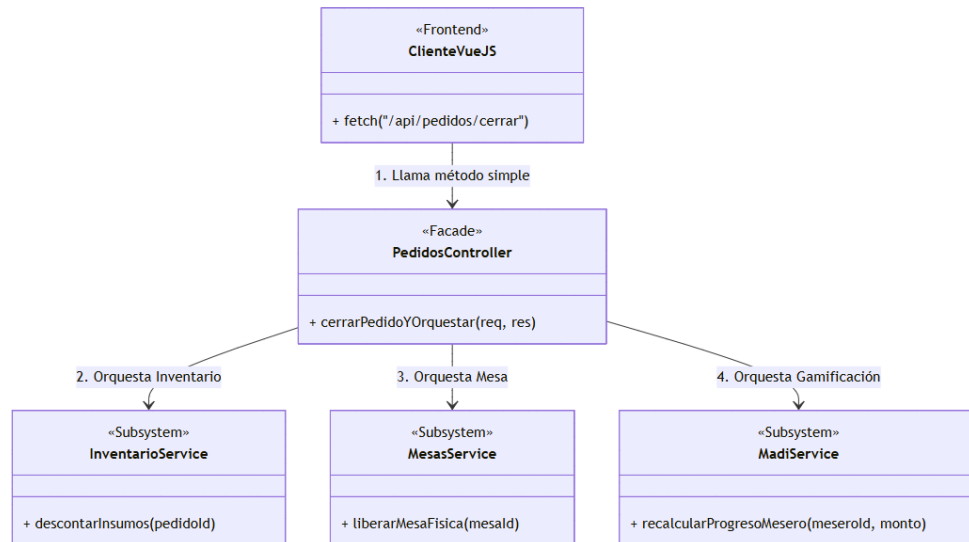
Diseño UML (Patrón Observer)



• Patrón Observer (Observador) mediante WebSockets)

Justificación: Procesos transaccionales como "Cobrar Pedido" en la cafetería requieren orquestar múltiples subsistemas a la vez: actualizar el estado del pedido, descontar stock del inventario, sumar la venta al desempeño del mesero y liberar la mesa física. Exponer esta complejidad al frontend es un riesgo de seguridad y viabilidad. Implementación: Se creó un controlador central (`pedidos.controller.js`) que actúa como Fachada. El Frontend hace una única petición REST simple (`POST /api/pedidos/cerrar`). Internamente, la Fachada orquesta las llamadas a todos los servicios de dominio necesarios y devuelve una respuesta consolidada.

Diseño UML (Patrón Observer)

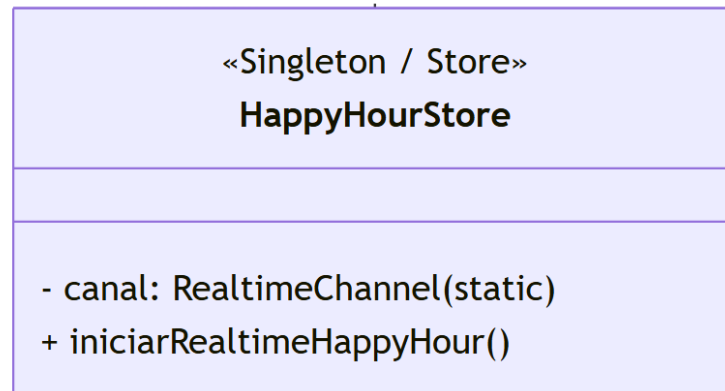


- **Patrón Singleton (Instancia Única) en Gestor de WebSockets**

Justificación: Al navegar entre diferentes pantallas reactivas (ej. cambiar de Mesero a Supervisor), los componentes intentaban reconectarse a los WebSockets (postgres_changes), lo que causaba colisiones ("cannot add callbacks after subscribe") y crasheos en la aplicación. Implementación: Se aplicó un patrón Singleton (o control de instancia única) en el store de Pinia (happyHourStore.js). Antes de intentar abrir un canal de Supabase Realtime, el sistema verifica si la variable canal ya tiene una conexión activa. Si ya existe, aborta la reconexión y reutiliza la instancia existente, garantizando una sola conexión global segura a lo largo de toda la sesión del usuario.

Diseño UML (Patrón Singleton)

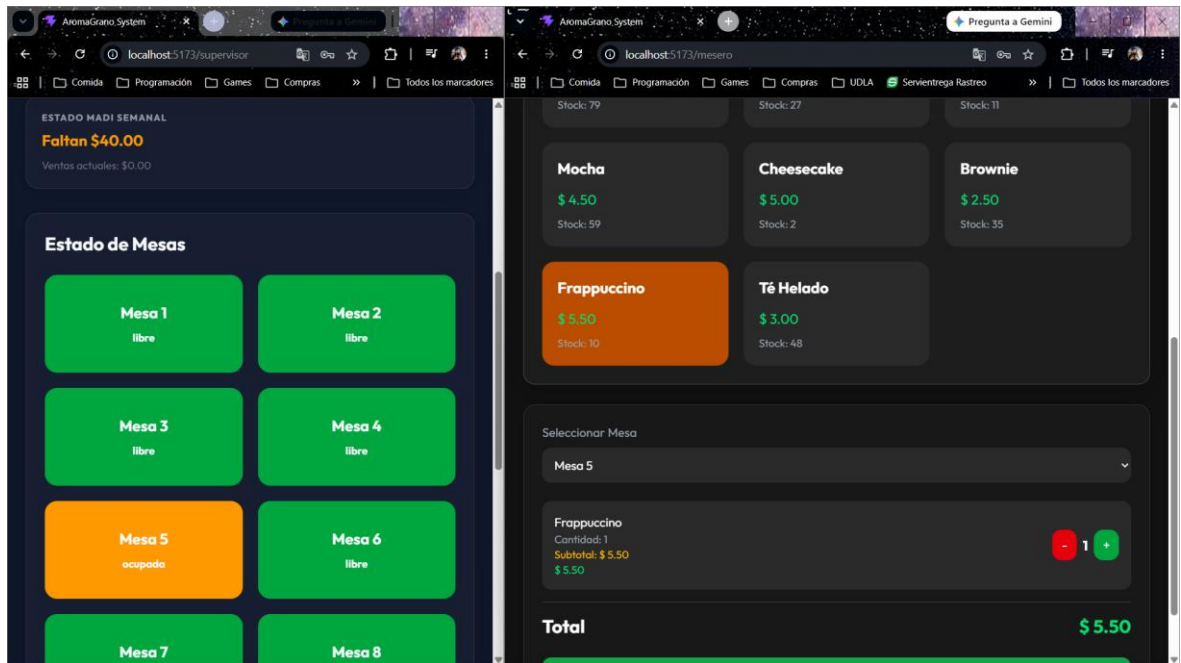
```
if (canal) return;\ncanal = supabase.channel(...).subscribe();
```



11. Pruebas Funcionales del Sistema

11.1. Prueba Funcional 1 (Patrón Observer): Sincronización de Mesas en Tiempo Real.

- **Objetivo:** Verificar que, al ocupar una mesa desde el POS del mesero, el cambio se refleje instantáneamente en el dashboard del Supervisor sin recargar la página.
- **Pasos ejecutados:**
 1. Se inicia sesión simultáneamente con rol `mesero@aromagrano.com` y `supervisor@aromagrano.com` en dos navegadores distintos.
 2. El mesero selecciona la Mesa 5 y abre un pedido.
- **Resultado Obtenido:** El dashboard del supervisor actualiza automáticamente la Mesa 5 al estado "Ocupada" mediante WebSockets.
- **Evidencia:**



11.2. Prueba Funcional 2 (Patrón Facade): Activación del Modo MADI / Happy Hour.

- **Objetivo:** Comprobar la orquestación del backend (Facade) y la reactividad del frontend al alcanzar la meta de ventas.
- **Pasos ejecutados:**
 1. Se registra y cierra un pedido que empuja las ventas totales del día por encima del umbral configurado por el administrador.
- **Resultado Obtenido:** El motor MORC valida la regla, Supabase emite el evento global, y la interfaz de todos los usuarios conectados transiciona al "Modo Neón" aplicando el multiplicador.
- **Evidencia:**

